DeepSeek Math and Prover

Deep Dive into DeepSeek — Chapter 10

Yin Yang

Foundation Books Project

The one rule of this chapter

A stronger checker changes *what* is certified.
It does not certify that the formal object matches the English one.

- An answer scorer, a symbolic checker, and a Lean proof assistant check progressively more structured objects.
- Moving up the ladder strengthens the oracle and raises the specification burden — it never repairs a mismatched formalization.
- DeepSeek-Math, DeepSeek-Prover, Prover-V1.5, Prover-V2, and Math-V2 occupy different rungs of one verifiability chain, not one undifferentiated "math model" category.

The one rule of this chapter

A stronger checker changes *what* is certified.
It does not certify that the formal object matches the English one.

- An answer scorer, a symbolic checker, and a Lean proof assistant check progressively more structured objects.
- Moving up the ladder strengthens the oracle and raises the specification burden — it never repairs a mismatched formalization.
- DeepSeek-Math, DeepSeek-Prover, Prover-V1.5, Prover-V2, and Math-V2 occupy different rungs of one verifiability chain, not one undifferentiated "math model" category.

Goal: carry one arithmetic equality from a boxed answer to a Lean-checked theorem, and see exactly what each verifier certifies along the way.

Roadmap

1. The verification ladder

Roadmap

1. The verification ladder
2. Running example: one equality, four objects

Roadmap

1. The verification ladder
2. Running example: one equality, four objects
3. DeepSeek-Math and rule-verifiable rewards

Roadmap

1. The verification ladder
2. Running example: one equality, four objects
3. DeepSeek-Math and rule-verifiable rewards
4. DeepSeek-Prover: verified data at scale

Roadmap

1. The verification ladder
2. Running example: one equality, four objects
3. DeepSeek-Math and rule-verifiable rewards
4. DeepSeek-Prover: verified data at scale
5. Prover-V1.5: Lean feedback inside search

Roadmap

1. The verification ladder
2. Running example: one equality, four objects
3. DeepSeek-Math and rule-verifiable rewards
4. DeepSeek-Prover: verified data at scale
5. Prover-V1.5: Lean feedback inside search
6. Prover-V2 and Math-V2: decomposition and self-verification

Roadmap

1. The verification ladder
2. Running example: one equality, four objects
3. DeepSeek-Math and rule-verifiable rewards
4. DeepSeek-Prover: verified data at scale
5. Prover-V1.5: Lean feedback inside search
6. Prover-V2 and Math-V2: decomposition and self-verification
7. Evaluation, failure, and reproducibility

The verification ladder

Four rungs, one running item

Rung	Checked object	Running-example check
Final answer	Extracted response value	Candidate equals 10

Four rungs, one running item

Rung	Checked object	Running-example check
Final answer	Extracted response value	Candidate equals 10
Symbolic	Expression or numeric value	$\lvert 36 - 26 \rvert = 10$

Four rungs, one running item

Rung	Checked object	Running-example check
Final answer	Extracted response value	Candidate equals 10
Symbolic	Expression or numeric value	$\lvert 36 - 26 \rvert = 10$
Program-aided	Executed computation	Python returns 10.0

Four rungs, one running item

Rung	Checked object	Running-example check
Final answer	Extracted response value	Candidate equals 10
Symbolic	Expression or numeric value	$\lvert 36 - 26 \rvert = 10$
Program-aided	Executed computation	Python returns 10.0
Proof assistant	Formal proof object	Lean theorem is accepted

Four rungs, one running item

Rung	Checked object	Running-example check
Final answer	Extracted response value	Candidate equals 10
Symbolic	Expression or numeric value	$\lvert 36 - 26 \rvert = 10$
Program-aided	Executed computation	Python returns 10.0
Proof assistant	Formal proof object	Lean theorem is accepted

- Moving down the table strengthens the oracle: it costs more to specify, and it certifies more.
- **Verifiability** (Def.) is the combination of a checked object, a verifier, and the scope that verifier can support.

Four rungs, one running item

Rung	Checked object	Running-example check
Final answer	Extracted response value	Candidate equals 10
Symbolic	Expression or numeric value	$\lvert 36 - 26 \rvert = 10$
Program-aided	Executed computation	Python returns 10.0
Proof assistant	Formal proof object	Lean theorem is accepted

- Moving down the table strengthens the oracle: it costs more to specify, and it certifies more.
- **Verifiability** (Def.) is the combination of a checked object, a verifier, and the scope that verifier can support.

Proof acceptance still leaves a separate audit: does the formal statement faithfully represent the English problem?

Where each branch sits on the ladder

Branch	Main ladder position	Verified scope
DeepSeek-Math	Final-answer, symbolic, tool-integrated checks	Informal or rule-verifiable response

Where each branch sits on the ladder

Branch	Main ladder position	Verified scope
DeepSeek-Math	Final-answer, symbolic, tool-integrated checks	Informal or rule-verifiable response
DeepSeek-Prover	Autoformalization, verified data, proof checking	Formal statement + accepted proof

Where each branch sits on the ladder

Branch	Main ladder position	Verified scope
DeepSeek-Math	Final-answer, symbolic, tool-integrated checks	Informal or rule-verifiable response
DeepSeek-Prover	Autoformalization, verified data, proof checking	Formal statement + accepted proof

- Prover-V1.5 adds Lean feedback *inside* the search loop.

Where each branch sits on the ladder

Branch	Main ladder position	Verified scope
DeepSeek-Math	Final-answer, symbolic, tool-integrated checks	Informal or rule-verifiable response
DeepSeek-Prover	Autoformalization, verified data, proof checking	Formal statement + accepted proof

- Prover-V1.5 adds Lean feedback *inside* the search loop.
- Prover-V2 adds recursive subgoal decomposition.

Where each branch sits on the ladder

Branch	Main ladder position	Verified scope
DeepSeek-Math	Final-answer, symbolic, tool-integrated checks	Informal or rule-verifiable response
DeepSeek-Prover	Autoformalization, verified data, proof checking	Formal statement + accepted proof

- Prover-V1.5 adds Lean feedback *inside* the search loop.
- Prover-V2 adds recursive subgoal decomposition.
- Math-V2 adds a learned verifier acting as a reward model.

Where each branch sits on the ladder

Branch	Main ladder position	Verified scope
DeepSeek-Math	Final-answer, symbolic, tool-integrated checks	Informal or rule-verifiable response
DeepSeek-Prover	Autoformalization, verified data, proof checking	Formal statement + accepted proof

- Prover-V1.5 adds Lean feedback *inside* the search loop.
- Prover-V2 adds recursive subgoal decomposition.
- Math-V2 adds a learned verifier acting as a reward model.

Five branches, five different checked objects — none of them interchangeable.

Running example: one equality, four objects

120% **of** 30 **minus** 130% **of** 20

$$\left| \frac{120}{100} \cdot 30 - \frac{130}{100} \cdot 20 \right| = 10.$$

- **English problem:** find the positive difference between the two percentages.

120% **of** 30 **minus** 130% **of** 20

$$\left|\frac{120}{100}\cdot 30-\frac{130}{100}\cdot 20\right|=10.$$

- **English problem:** find the positive difference between the two percentages.
- **Numeric answer:** 10, extractable from a boxed final value.

120% **of** 30 **minus** 130% **of** 20

$$\left|\frac{120}{100}\cdot 30-\frac{130}{100}\cdot 20\right|=10.$$

- **English problem:** find the positive difference between the two percentages.
- **Numeric answer:** 10, extractable from a boxed final value.
- **Formal theorem:** the same equality, over $\mathbb{R}$, with `Mathlib` and `Aesop` imported.

120% **of** 30 **minus** 130% **of** 20

$$\left|\frac{120}{100}\cdot 30-\frac{130}{100}\cdot 20\right|=10.$$

- **English problem:** find the positive difference between the two percentages.
- **Numeric answer:** 10, extractable from a boxed final value.
- **Formal theorem:** the same equality, over $\mathbb{R}$, with `Mathlib` and `Aesop` imported.
- **Formalization audit:** did the formal statement keep the percentages, the absolute value, and the ordering?

120% **of** 30 **minus** 130% **of** 20

$$\left|\frac{120}{100}\cdot 30-\frac{130}{100}\cdot 20\right|=10.$$

- **English problem:** find the positive difference between the two percentages.
- **Numeric answer:** 10, extractable from a boxed final value.
- **Formal theorem:** the same equality, over $\mathbb{R}$, with `Mathlib` and `Aesop` imported.
- **Formalization audit:** did the formal statement keep the percentages, the absolute value, and the ordering?

These four objects are related, but no one of them is identical to another — that is the running thread of the whole chapter.

From concept to code: the formal target

The Prover-V2 README's style of Lean-style theorem for this exact problem has a companion fixture in this book's code release.

from `code/chapter_deepseek_math_and_prover/fixtures/percentage_difference.lean`

```
import Mathlib
import Aesop

theorem percentage_difference :
    abs (((120 : Real) / 100) * 30 -
         ((130 : Real) / 100) * 20) = 10 := by
  norm_num [abs_of_nonneg]
```

From concept to code: the formal target

The Prover-V2 README's style of Lean-style theorem for this exact problem has a companion fixture in this book's code release.

from `code/chapter_deepseek_math_and_prover/fixtures/percentage_difference.lean`

```
import Mathlib
import Aesop

theorem percentage_difference :
    abs (((120 : Real) / 100) * 30 -
         ((130 : Real) / 100) * 20) = 10 := by
  norm_num [abs_of_nonneg]
```

`norm_num [abs_of_nonneg]` either leaves zero goals — a checked proof — or it does not. There is no partial credit at the kernel. [INSPECTED: book fixture, `percentage_difference.lean`]

DeepSeek-Math and rule-verifiable rewards

A reward that reads only the boxed answer

- DeepSeek-Math starts from DeepSeek-Coder-v1.5 7B and continues training on math, natural-language, and code data [REPORTED: DeepSeek-Math paper / repository README].

A reward that reads only the boxed answer

- DeepSeek-Math starts from DeepSeek-Coder-v1.5 7B and continues training on math, natural-language, and code data [REPORTED: DeepSeek-Math paper / repository README].
- Its evaluation configs name process, extraction, and evaluation functions for GSM8K, MATH, and miniF2F-Isabelle.

A reward that reads only the boxed answer

- DeepSeek-Math starts from DeepSeek-Coder-v1.5 7B and continues training on math, natural-language, and code data [REPORTED: DeepSeek-Math paper / repository README].
- Its evaluation configs name process, extraction, and evaluation functions for GSM8K, MATH, and miniF2F-Isabelle.
- An exact-answer verifier $V(q, y) \in \{0, 1, \text{error}\}$ extracts a candidate and compares it against the accepted set.

A reward that reads only the boxed answer

- DeepSeek-Math starts from DeepSeek-Coder-v1.5 7B and continues training on math, natural-language, and code data [REPORTED: DeepSeek-Math paper / repository README].
- Its evaluation configs name process, extraction, and evaluation functions for GSM8K, MATH, and miniF2F-Isabelle.
- An exact-answer verifier $V(q, y) \in \{0, 1, \text{error}\}$ extracts a candidate and compares it against the accepted set.

The reward boundary is only the extracted object. A trace that says "130% of 20 is 24" can still end in $\boxed{10}$ and score 1 — correct answer, defective reasoning, both true at once.

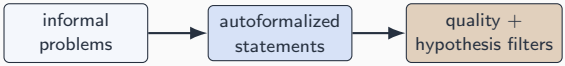
DeepSeek-Prover: verified data at scale

Autoformalize, filter, prove, recycle

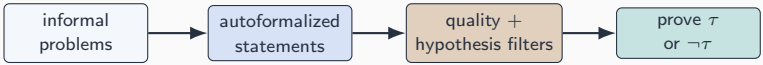
informal problems

Autoformalize, filter, prove, recycle

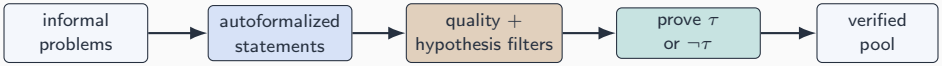
Autoformalize, filter, prove, recycle



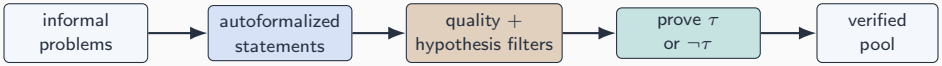
Autoformalize, filter, prove, recycle



Autoformalize, filter, prove, recycle

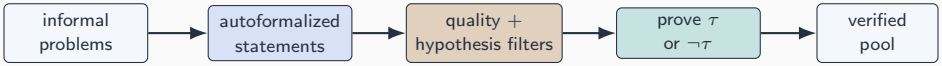


Autoformalize, filter, prove, recycle



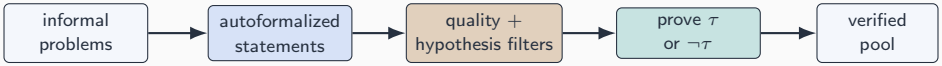
- Started from 869,659 informal problems; the quality filter leaves 712,073 statements before proof search [REPORTED: original DeepSeek-Prover paper].

Autoformalize, filter, prove, recycle



- Started from 869,659 informal problems; the quality filter leaves 712,073 statements before proof search [REPORTED: original DeepSeek-Prover paper].
- A hypothesis-rejection test catches vacuous statements whose hypotheses already entail `False`. Proving τ *or* $\neg\tau$ still yields a usable checker record either way.

Autoformalize, filter, prove, recycle



- Started from 869,659 informal problems; the quality filter leaves 712,073 statements before proof search [REPORTED: original DeepSeek-Prover paper].
- A hypothesis-rejection test catches vacuous statements whose hypotheses already entail `False`. Proving τ *or* $\neg\tau$ still yields a usable checker record either way.

Acceptance certifies the recorded formal branch, never semantic equivalence with the informal problem.

Prover-V1.5: Lean feedback inside search

From concept to code: checker fields become search state

RMaxTS keeps a tree of proof states and reads structured Lean output to decide what to expand next.

from `DeepSeek-Prover-V1.5/prover/lean/verifier.py`

```
result['pass'] = not result['errors']
result['complete'] = (result['pass'] and not result['sorries']
    and not any("declaration uses 'sorry'" in w['data'] for w in result['warnings']))
```

From concept to code: checker fields become search state

RMaxTS keeps a tree of proof states and reads structured Lean output to decide what to expand next.

from `DeepSeek-Prover-V1.5/prover/lean/verifier.py`

```
result['pass'] = not result['errors']
result['complete'] = (result['pass'] and not result['sorries']
    and not any("declaration uses 'sorry'" in w['data'] for w in result['warnings']))
```

`pass` and `complete` are different questions: a script can parse (`pass`) while a goal or `sorry` remains (`complete=false`). Only `pass and complete` is a proof success. [INSPECTED: DeepSeek-Prover-V1.5 `prover/lean/verifier.py`]

Prover-V2 and Math-V2: decomposition and self-verification

Two more ways to strengthen the object

Prover-V2: subgoal decomposition

- Rewrites τ into subgoals $\tau_1, \ldots, \tau_m$ plus a composition obligation.
- DeepSeek-V3 decomposes; a 7B prover closes each subgoal.
- Solved subgoals earn credit only when the composition itself checks.

Math-V2: self-verifiable reasoning

- Trains a proof verifier, then uses it as a reward model for the generator.
- Scales verification compute to label hard-to-verify proofs.
- Generator and verifier feedback stay two separate evidence trails.

Two more ways to strengthen the object

Prover-V2: subgoal decomposition

- Rewrites τ into subgoals $\tau_1, \ldots, \tau_m$ plus a composition obligation.
- DeepSeek-V3 decomposes; a 7B prover closes each subgoal.
- Solved subgoals earn credit only when the composition itself checks.

Math-V2: self-verifiable reasoning

- Trains a proof verifier, then uses it as a reward model for the generator.
- Scales verification compute to label hard-to-verify proofs.
- Generator and verifier feedback stay two separate evidence trails.

Reported: Prover-V2-671B reaches 88.9% on miniF2F-test and solves 49/658 PutnamBench problems [REPORTED: DeepSeek-Prover-V2 paper].

Evaluation, failure, and reproducibility

Same equality, different denominators

- Informal answer accuracy counts accepted normalized answers over evaluated items under a named sampling rule.

Same equality, different denominators

- Informal answer accuracy counts accepted normalized answers over evaluated items under a named sampling rule.
- Whole-proof success counts theorems with at least one kernel-accepted proof over attempted theorems under a named budget.

Same equality, different denominators

- Informal answer accuracy counts accepted normalized answers over evaluated items under a named sampling rule.
- Whole-proof success counts theorems with at least one kernel-accepted proof over attempted theorems under a named budget.
- A timeout is a budget stop, not a mathematical rejection; an accepted proof still needs a formalization audit.

Same equality, different denominators

- Informal answer accuracy counts accepted normalized answers over evaluated items under a named sampling rule.
- Whole-proof success counts theorems with at least one kernel-accepted proof over attempted theorems under a named budget.
- A timeout is a budget stop, not a mathematical rejection; an accepted proof still needs a formalization audit.

Quoting a benchmark row without its protocol — dataset, split, checker, budget — removes the information needed to compare it to anything else.

Concept-to-code map for the chapter

- **boxed-answer prompt and evaluation configs** → `DeepSeek-Math/evaluation/configs/zero_shot_test_configs.json`

Concept-to-code map for the chapter

- **boxed-answer prompt and evaluation configs** $\rightarrow$
  `DeepSeek-Math/evaluation/configs/zero_shot_test_configs.json`
- **Lean checker fields (pass, complete, sorries)** $\rightarrow$
  `DeepSeek-Prover-V1.5/prover/lean/verifier.py`

Concept-to-code map for the chapter

- **boxed-answer prompt and evaluation configs** → `DeepSeek-Math/evaluation/configs/zero_shot_test_configs.json`
- **Lean checker fields (pass, complete, sorries)** → `DeepSeek-Prover-V1.5/prover/lean/verifier.py`
- **RMaxTS search state and budget** → `DeepSeek-Prover-V1.5/configs/RMaxTS.py`

Concept-to-code map for the chapter

- **boxed-answer prompt and evaluation configs** → `DeepSeek-Math/evaluation/configs/zero_shot_test_configs.json`
- **Lean checker fields (pass, complete, sorries)** → `DeepSeek-Prover-V1.5/prover/lean/verifier.py`
- **RMaxTS search state and budget** → `DeepSeek-Prover-V1.5/configs/RMaxTS.py`
- **learned-verifier prompt templates** → `DeepSeek-Math-V2/inference/math_templates.py`

Concept-to-code map for the chapter

- **boxed-answer prompt and evaluation configs** → `DeepSeek-Math/evaluation/configs/zero_shot_test_configs.json`
- **Lean checker fields (pass, complete, sorries)** → `DeepSeek-Prover-V1.5/prover/lean/verifier.py`
- **RMaxTS search state and budget** → `DeepSeek-Prover-V1.5/configs/RMaxTS.py`
- **learned-verifier prompt templates** → `DeepSeek-Math-V2/inference/math_templates.py`
- **the running theorem, checked** → `code/chapter_deepseek_math_and_prover/fixtures/percentage_difference.lean`

Concept-to-code map for the chapter

- **boxed-answer prompt and evaluation configs** → `DeepSeek-Math/evaluation/configs/zero_shot_test_configs.json`
- **Lean checker fields (pass, complete, sorries)** → `DeepSeek-Prover-V1.5/prover/lean/verifier.py`
- **RMaxTS search state and budget** → `DeepSeek-Prover-V1.5/configs/RMaxTS.py`
- **learned-verifier prompt templates** → `DeepSeek-Math-V2/inference/math_templates.py`
- **the running theorem, checked** → `code/chapter_deepseek_math_and_prover/fixtures/percentage_difference.lean`

Certify the object you actually checked, and say so on the slide.

Next: the reasoning-RL chapters (R1-Zero, R1) generalize the rule-verifiable reward this chapter specializes to math and proofs.